Lab 7: Fuzzing for Input Validation Bugs

Name: Satyam Goyal
SRN: PES1UG24CS424
Module under test: phone_repair_quote_parser.py (mask_imei, parse_parts_order, parse_model)
Tool: Hypothesis (property-based fuzzing) with pytest

Results at a glance

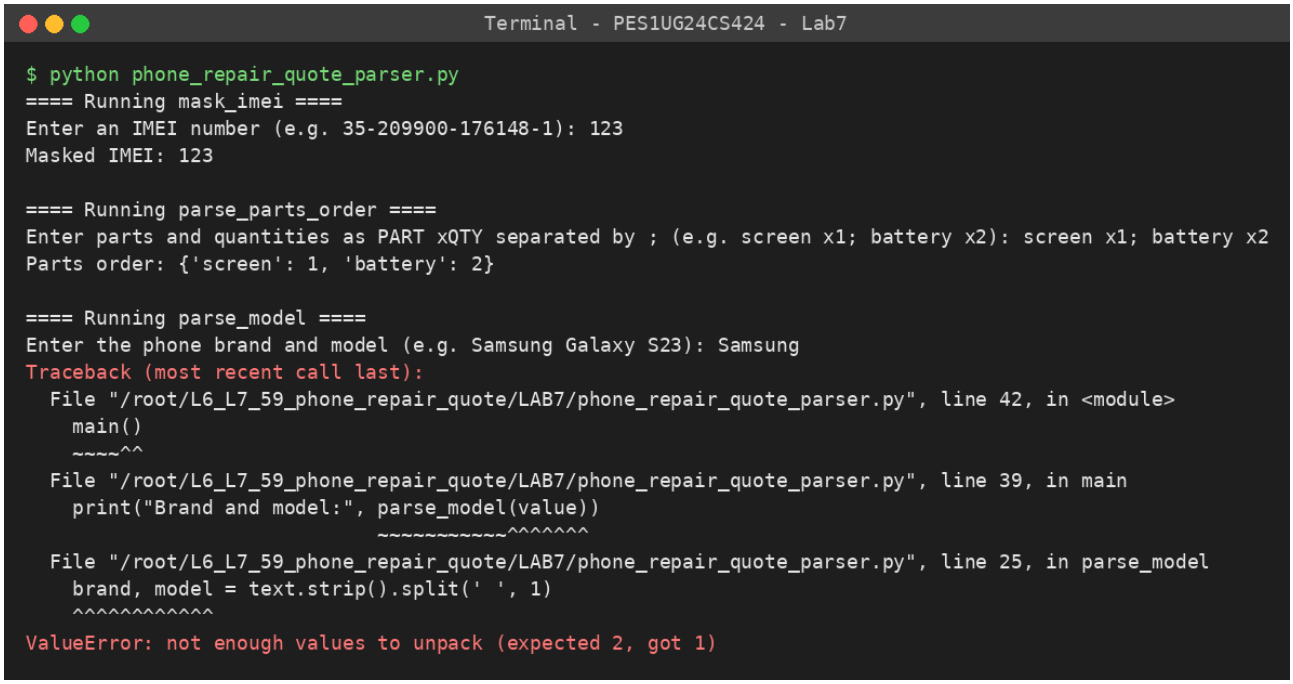
Screenshot	Command	Result
SS1	python phone_repair_quote_parser.py (buggy)	Crashes with ValueError on a one-word phone model; also prints the short IMEI 123 unmasked
SS2	pytest test_phone_repair_quote_parser.py (buggy)	16 failed, 1 passed
SS3	python phone_repair_quote_parser.py (fixed)	Same inputs run cleanly, IMEI masked, no crash
SS4	pytest test_phone_repair_quote_parser.py (fixed)	17 passed

Contents

1. Buggy module: SS1 and SS2
2. Bugs found and fixes
3. Fixed module: SS3 and SS4
4. Reflection answers
Appendix A: fixed module code
Appendix B: completed test code

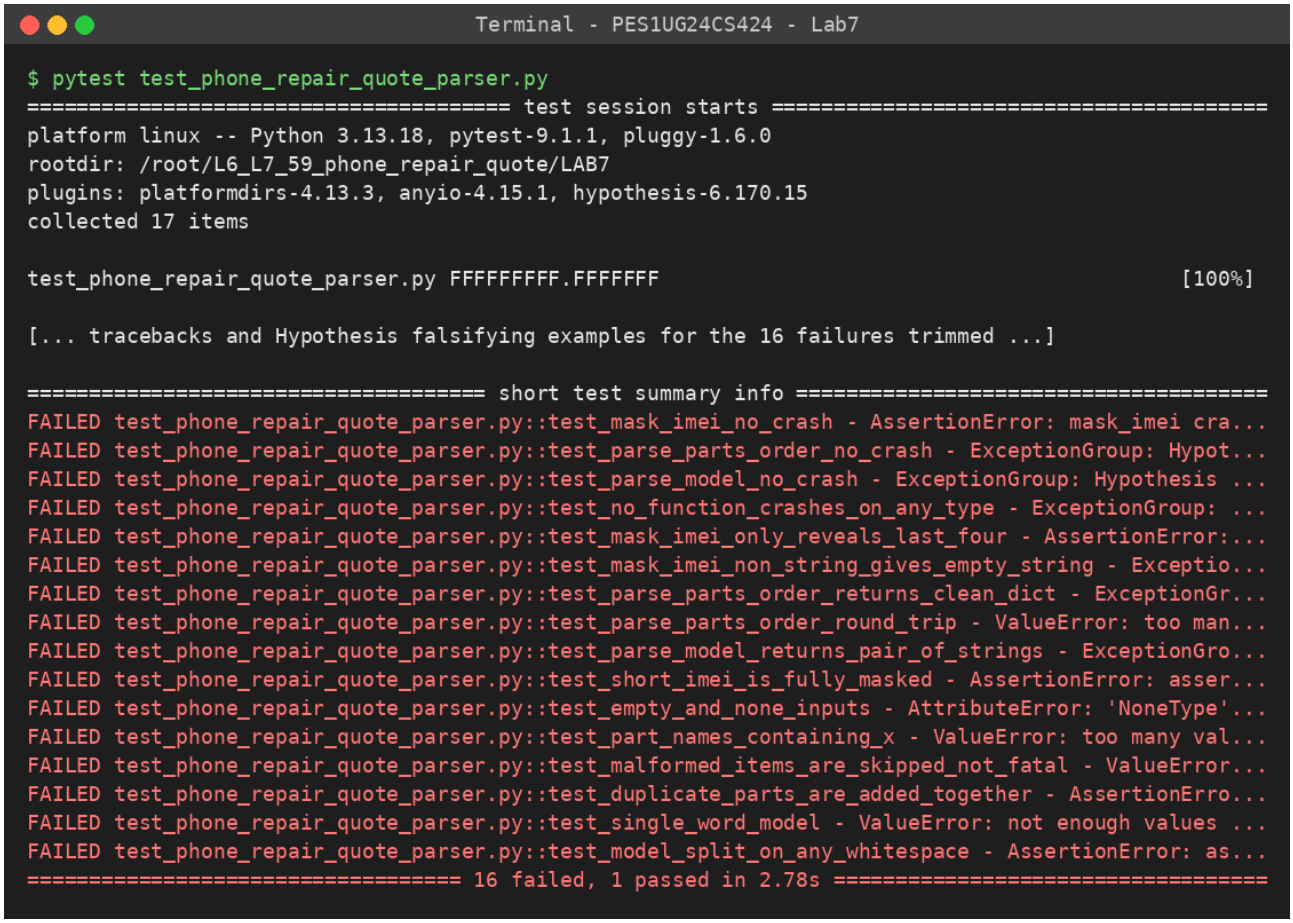
1. Step 2: Running the buggy module (SS1)

Command: `python phone_repair_quote_parser.py`. The IMEI 123 comes back unmasked, the parts order works, and the one-word model Samsung crashes the program.



Step 4: Running the Hypothesis tests on the buggy module (SS2)

Command: `pytest test_phone_repair_quote_parser.py`. 16 of 17 tests fail. The output is trimmed to the session header and the summary; each failure also logs the smallest input Hypothesis found (listed in section 2).


Terminal - PES1UG24CS424 - Lab7
$ pytest test_phone_repair_quote_parser.py
========== test session starts ==========
platform linux -- Python 3.13.18, pytest-9.1.1, pluggy-1.6.0
rootdir: /root/L6_L7_59_phone_repair_quote/LAB7
plugins: platformdirs-4.13.3, anyio-4.15.1, hypothesis-6.170.15
collected 17 items
test_phone_repair_quote_parser.py FFFFFFFFF.FFFFFFF [100%]
[... tracebacks and Hypothesis falsifying examples for the 16 failures trimmed ...]
========== short test summary info ==========
FAILED test_phone_repair_quote_parser.py::test_mask_imei_no_crash - AssertionError: mask_imei cra...
FAILED test_phone_repair_quote_parser.py::test_parse_parts_order_no_crash - ExceptionGroup: Hypot...
FAILED test_phone_repair_quote_parser.py::test_parse_model_no_crash - ExceptionGroup: Hypothesis ...
FAILED test_phone_repair_quote_parser.py::test_no_function_crashes_on_any_type - ExceptionGroup: ...
FAILED test_phone_repair_quote_parser.py::test_mask_imei_only_reveals_last_four - AssertionError:...
FAILED test_phone_repair_quote_parser.py::test_mask_imei_non_string_gives_empty_string - Exceptio...
FAILED test_phone_repair_quote_parser.py::test_parse_parts_order_returns_clean_dict - ExceptionGr...
FAILED test_phone_repair_quote_parser.py::test_parse_parts_order_round_trip - ValueError: too man...
FAILED test_phone_repair_quote_parser.py::test_parse_model_returns_pair_of_strings - ExceptionGro...
FAILED test_phone_repair_quote_parser.py::test_short_imei_is_fully_masked - AssertionError: asser...
FAILED test_phone_repair_quote_parser.py::test_empty_and_none_inputs - AttributeError: 'NoneType'...
FAILED test_phone_repair_quote_parser.py::test_part_names_containing_x - ValueError: too many val...
FAILED test_phone_repair_quote_parser.py::test_malformed_items_are_skipped_not_fatal - ValueError...
FAILED test_phone_repair_quote_parser.py::test_duplicate_parts_are_added_together - AssertionErro...
FAILED test_phone_repair_quote_parser.py::test_single_word_model - ValueError: not enough values ...
FAILED test_phone_repair_quote_parser.py::test_model_split_on_any_whitespace - AssertionError: as...
========== 16 failed, 1 passed in 2.78s ==========

2. Bugs found and fixes

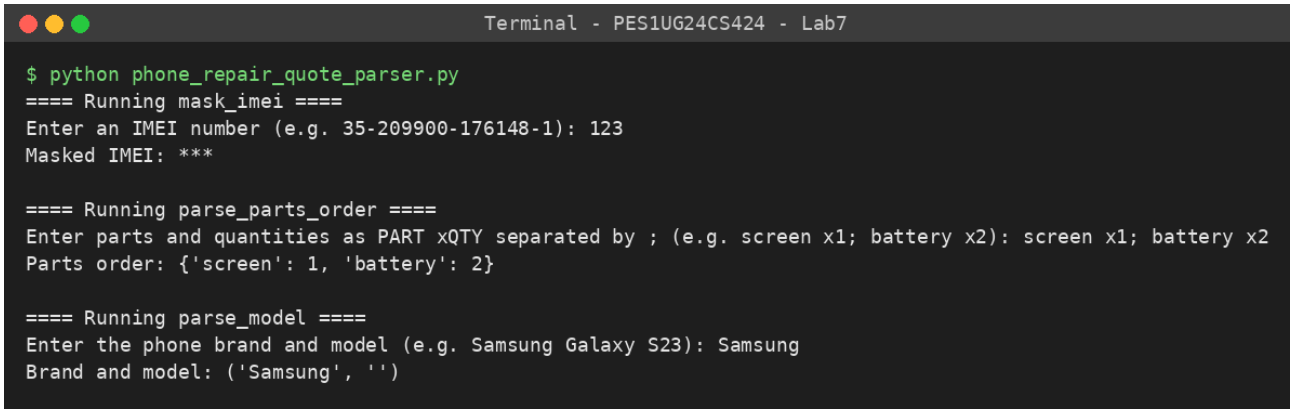
Unmarked failing inputs are the minimal ones Hypothesis reported after shrinking. Rows marked (manual) come from my own regression tests, which cover cases that random text rarely reaches. 'Samsung' is the input used in SS1.

Function	Failing input	What went wrong	Fix
mask_imei	None, or b'' (bytes)	AttributeError / TypeError: the code calls .replace on whatever it is given	Return "" when the value is not a string
mask_imei	'0' (any value of 4 or fewer characters)	No crash, but the whole number is returned unmasked. '*' * negative is '' and [-4:] keeps everything. This leaks the IMEI.	Mask every character when 4 or fewer remain
parse_parts_order	'' (empty text)	''.split('x') gives one piece, so unpacking into part, qty raises ValueError	Strip each item and skip empty ones
parse_parts_order	'x'	int('') raises ValueError: no quantity after the x	Require a trailing digit run and a part name; skip bad items
parse_parts_order	None	AttributeError: None has no .split	Return {} when not a string
parse_parts_order	'toolbox x3' (manual)	split('x') cuts at the x inside the part name, ValueError: too many values to unpack	Split off the trailing digits first, then take the last x as the marker
parse_parts_order	'charger x-1', 'camera xabc' (manual)	Negative quantities accepted; non numeric crashes	Only positive whole quantities accepted, others skipped
parse_parts_order	'screen x1; screen x2' (manual)	Second entry silently overwrites the first, so 3 screens become 2	Add quantities for repeated parts
parse_model	'' or 'Samsung'	split(' ', 1) gives one piece, ValueError on unpacking	Split on whitespace, return (brand, "") for one word and ("", "") for blank
parse_model	None	AttributeError: None has no .strip	Return ("", "") when not a string
parse_model	'apple\tiPhone 15' (manual)	Only a plain space separates brand and model, so a tab ends up in the brand	split(None, 1) splits on any whitespace

Design choice: the handout asks that the functions "handle any input", so invalid input is neutralised (empty result, malformed items skipped) instead of raising. An alternative is to raise a clear ValueError for bad input; that would be a deliberate error, but it would fail the no-crash tests as written.

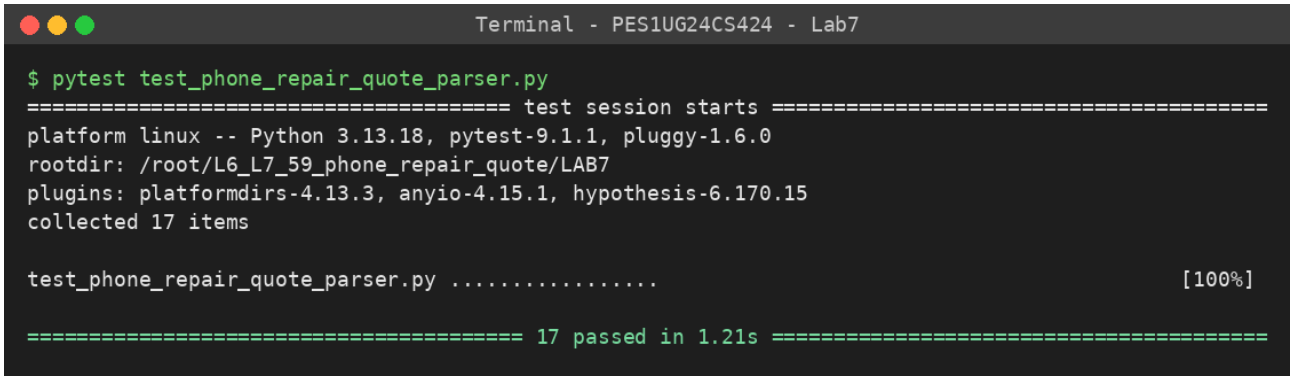
3. Step 5: Running the fixed module (SS3)

Command: `python phone_repair_quote_parser.py` with the same three inputs as SS1. IMEI 123 is now fully masked and Samsung no longer crashes.



Step 6: Re-running the tests on the fixed module (SS4)

Command: `pytest test_phone_repair_quote_parser.py`. All 17 tests pass. As an extra check the tests were also run with five different Hypothesis seeds and the three functions were fed 50,000 mixed-type random inputs; there were no failures or crashes.



4. Reflection answers

Note: the handout text I worked from does not list the reflection questions, so these are the three questions I answered: what was found, how it was fixed, and why fuzzing helps.

Q1. What bugs did fuzzing find, and which inputs triggered them?

All three functions trusted their input. Hypothesis found crashes on None (AttributeError) in every function, on bytes input (TypeError), and on empty or one-word text (ValueError from unpacking) in parse_parts_order and parse_model. It also found that parse_parts_order crashes on 'x' (int of an empty string) and that mask_imei('0') returns the number unmasked, which is a silent data leak rather than a crash. My own regression tests added the part name toolbox (the x inside it breaks split), negative or non numeric quantities, duplicate parts losing quantity, and a tab between brand and model.

Q2. How did you fix them?

Each function now checks the type first and returns a safe empty value for non-strings. mask_imei masks everything when four or fewer characters remain. parse_parts_order strips and skips empty items, reads the quantity as the trailing digits, requires an x marker and a part name before it, ignores items that do not fit, and adds up repeated parts. parse_model splits on any whitespace and handles blank and one-word text. The same tests that failed in SS2 now pass in SS4.

Q3. Why is fuzzing better than hand-written unit tests here, and what are its limits?

Hand-picked examples written from the docstrings assume well formed text, so they would pass while the code still crashes on '' or None. A fuzzer has no such assumption: by default Hypothesis tries up to 100 generated inputs per test, including empty strings, odd Unicode and other types, and shrinks each failure to a minimal case. Its limits: a no-crash property only proves the code does not raise, not that the result is right, so I added properties for correct output (masking, round trip, clean dictionary). Fuzzing also cannot find a bug in behaviour nobody described, such as whether duplicate parts should be added or the last one should win; that needed a human decision.

Appendix A: phone_repair_quote_parser.py (fixed)

```
DIGITS = "0123456789"


def mask_imei(imei):
    """
    Removes dashes and spaces, then hides all but the last four digits.

    Never raises. Returns "" when imei is not a string. When four or fewer
    characters remain, every one of them is masked, because showing "the last
    four" of such a value would reveal the whole number.
    """
    if not isinstance(imei, str):
        return ""
    imei = imei.replace('-', '').replace(' ', '')
    if len(imei) <= 4:
        return '*' * len(imei)
    return '*' * (len(imei) - 4) + imei[-4:]


def parse_parts_order(text):
    """
    Parses 'screen x1; battery x2' into a {part: quantity} dictionary.

    Never raises. Returns {} when text is not a string. Items that are empty
    or not written as 'part xQTY' (missing part, missing or non numeric
    quantity, negative or zero quantity) are skipped. Part names may contain
    the letter x, and repeated parts have their quantities added together.
    """
    order = {}
    if not isinstance(text, str):
        return order
    for item in text.split(';'):
        item = item.strip()
        # split off the trailing run of digits (the quantity)
        i = len(item)
        while i > 0 and item[i - 1] in DIGITS:
            i -= 1
        qty_text, rest = item[i:], item[:i].rstrip()
        # what is left must end with the 'x' marker, preceded by a part name
        if not qty_text or not rest or rest[-1] not in 'xX':
            continue
        part = rest[:-1].strip()
        try:
            qty = int(qty_text)
        except ValueError:      # absurdly long digit strings
            continue
        if not part or qty < 1:
            continue
        order[part] = order.get(part, 0) + qty
    return order


def parse_model(text):
    """
    Splits 'Brand Model name' into a (brand, model) tuple.

    Never raises. Returns ("", "") for non-strings and blank text, and
    (brand, "") when only one word is given. Any whitespace (spaces, tabs)
    separates the brand from the model.
    """
    if not isinstance(text, str):
        return "", ""
    parts = text.strip().split(None, 1)
    if not parts:
        return "", ""
    brand = parts[0].title()
    model = parts[1].strip() if len(parts) > 1 else ""
    return brand, model


def main():
    print("==== Running mask_imei ====")
    value = input("Enter an IMEI number (e.g. 35-209900-176148-1): ")
    print("Masked IMEI:", mask_imei(value))

    print("\n==== Running parse_parts_order ====")
    value = input("Enter parts and quantities as PART xQTY separated by ; (e.g. screen x1; battery x2): ")
    print("Parts order:", parse_parts_order(value))

    print("\n==== Running parse_model ====")
```

```
    value = input("Enter the phone brand and model (e.g. Samsung Galaxy S23): ")
    print("Brand and model:", parse_model(value))

if __name__ == "__main__":
    main()
```

Appendix B: test_phone_repair_quote_parser.py (completed)

```
"""
Lab 7 - Fuzz tests (Hypothesis) for phone_repair_quote_parser.py
Name : Satyam Goyal
SRN  : PES1UG24CS424

Run:
    pytest test_phone_repair_quote_parser.py
    python -m pytest test_phone_repair_quote_parser.py
"""
import string

from hypothesis import given, strategies as st
from phone_repair_quote_parser import mask_imei, parse_parts_order, parse_model


# ---------------------------------------------------------------------------
# Part 1: required "must not crash" fuzz tests (one per function)
# ---------------------------------------------------------------------------
@given(st.text() | st.none())
def test_mask_imei_no_crash(s):
    try:
        mask_imei(s)
    except Exception as e:
        assert False, f"mask_imei crashed with: {e!r}"


@given(st.text() | st.none())
def test_parse_parts_order_no_crash(s):
    try:
        parse_parts_order(s)
    except Exception as e:
        assert False, f"parse_parts_order crashed with: {e!r}"


@given(st.text() | st.none())
def test_parse_model_no_crash(s):
    try:
        parse_model(s)
    except Exception as e:
        assert False, f"parse_model crashed with: {e!r}"


# ---------------------------------------------------------------------------
# Part 2: no crash for values of any type, not only str and None
# ---------------------------------------------------------------------------
any_value = st.one_of(
    st.none(),
    st.booleans(),
    st.integers(),
    st.floats(),
    st.binary(),
    st.text(),
    st.lists(st.integers()),
    st.dictionaries(st.text(), st.integers()),
)


@given(any_value)
def test_no_function_crashes_on_any_type(x):
    for fn in (mask_imei, parse_parts_order, parse_model):
        try:
            fn(x)
        except Exception as e:
            assert False, f"{fn.__name__} crashed on {x!r} with: {e!r}"


# ---------------------------------------------------------------------------
# Part 3: correctness properties (the output must also make sense)
# ---------------------------------------------------------------------------
@given(st.text())
def test_mask_imei_only_reveals_last_four(s):
    cleaned = s.replace("-", "").replace(" ", "")
    result = mask_imei(s)
```

```
    assert isinstance(result, str)
    assert len(result) == len(cleaned)
    if len(cleaned) > 4:
        assert result == "*" * (len(cleaned) - 4) + cleaned[-4:]
    else:
        # nothing may be revealed when there are four or fewer digits
        assert result == "*" * len(cleaned)


@given(st.one_of(st.none(), st.integers(), st.binary()))
def test_mask_imei_non_string_gives_empty_string(x):
    assert mask_imei(x) == ""


@given(st.text() | st.none())
def test_parse_parts_order_returns_clean_dict(s):
    result = parse_parts_order(s)
    assert isinstance(result, dict)
    for part, qty in result.items():
        assert isinstance(part, str) and part == part.strip() and part != ""
        assert isinstance(qty, int) and qty >= 1


part_names = (
    st.text(alphabet=string.ascii_letters + " ", min_size=1, max_size=15)
    .map(str.strip)
    .filter(bool)
)


@given(st.dictionaries(part_names, st.integers(min_value=1, max_value=10**6), min_size=1, max_size=6))
def test_parse_parts_order_round_trip(order):
    text = "; ".join(f"{part} x{qty}" for part, qty in order.items())
    assert parse_parts_order(text) == order


@given(st.text() | st.none())
def test_parse_model_returns_pair_of_strings(s):
    result = parse_model(s)
    assert isinstance(result, tuple) and len(result) == 2
    assert all(isinstance(piece, str) for piece in result)


# ---------------------------------------------------------------------------
# Part 4: regression tests for the kinds of input the fuzzer found
# ---------------------------------------------------------------------------
def test_normal_inputs_still_work():
    assert mask_imei("35-209900-176148-1") == "*" * 11 + "1481"
    assert parse_parts_order("screen x1; battery x2") == {"screen": 1, "battery": 2}
    assert parse_model("samsung Galaxy S23") == ("Samsung", "Galaxy S23")


def test_short_imei_is_fully_masked():
    assert mask_imei("123") == "***"


def test_empty_and_none_inputs():
    assert mask_imei("") == ""
    assert mask_imei(None) == ""
    assert parse_parts_order("") == {}
    assert parse_parts_order(None) == {}
    assert parse_model("") == ("", "")
    assert parse_model(None) == ("", "")


def test_part_names_containing_x():
    assert parse_parts_order("toolbox x3; xenon lamp x1") == {"toolbox": 3, "xenon lamp": 1}


def test_malformed_items_are_skipped_not_fatal():
    assert parse_parts_order("screen x1;; battery; camera xabc; x2; charger x-1") == {"screen": 1}


def test_duplicate_parts_are_added_together():
    assert parse_parts_order("screen x1; screen x2") == {"screen": 3}


def test_single_word_model():
    assert parse_model("samsung") == ("Samsung", "")


def test_model_split_on_any_whitespace():
    assert parse_model("apple\tiPhone 15") == ("Apple", "iPhone 15")
```